Mohaiya aldossary
Student ID: 2230004579

In [1]:
```
%pip install -q pandas scikit-learn nltk gensim
```

```
Note: you may need to restart the kernel to use updated packages.
[notice] A new release of pip is available: 23.2.1 -> 26.2.1
[notice] To update, run: C:\Users\ezio9\OneDrive\Desktop\IAU\NLP\NLP lab\.lab_work\venv\Scripts\python.exe -m pip install --upgrade pip
```

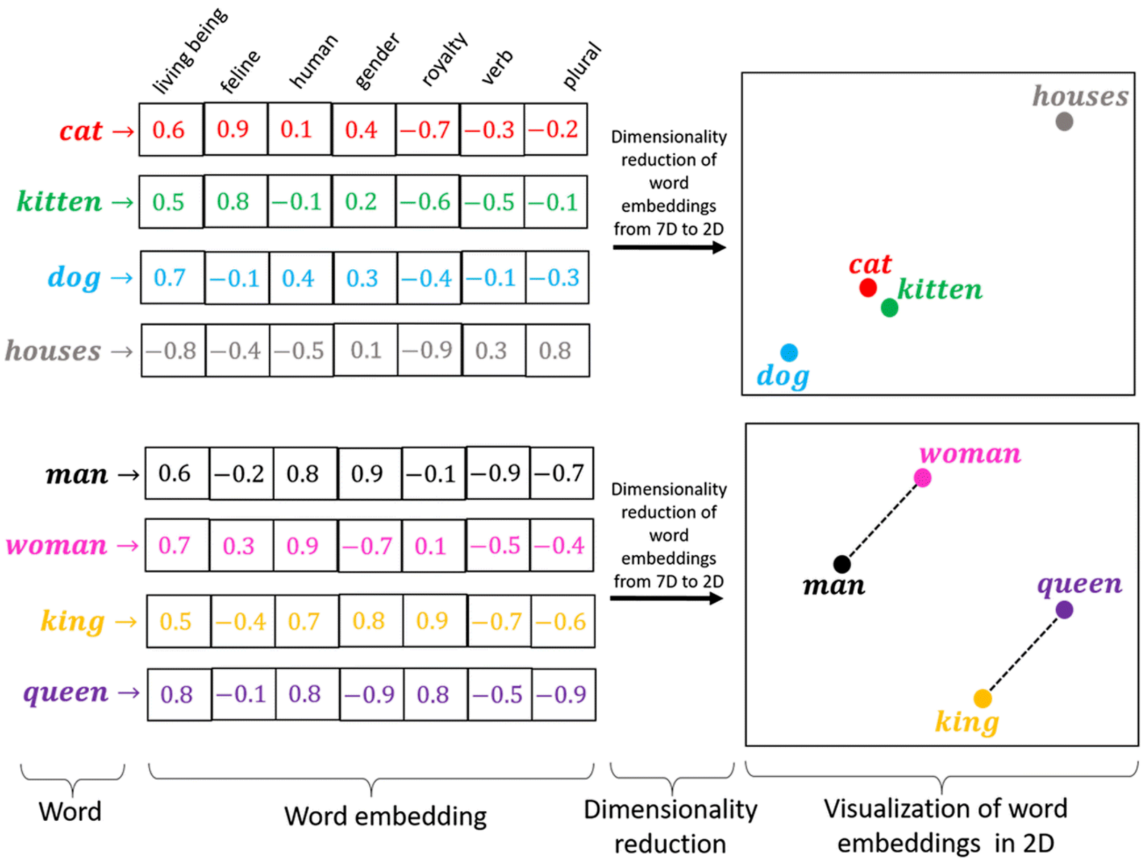
Text Representation

Text representation in NLP means converting text into a numerical form that a machine learning model can understand and process.

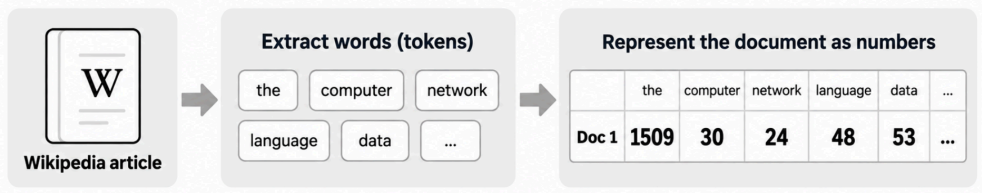
What is Embedding?

Embeddings in NLP is a technique where individual words are represented as real-valued vectors and captures inter-word semantics.

In this notebook, We going to intreduce 2 techniques for embedding. These techniques will be used for a machine learning models such as SVM, Random forest, ... ect.



1- TF-IDF



TF-IDF stands for term frequency-inverse document frequency. It is a measure that discounts common words. Used in the fields of information retrieval (IR) and machine learning, that can quantify the importance of words in a document amongst a collection of documents (also known as a corpus).

Components of TF-IDF

1. **TF (Term Frequency):**
   Measures how frequently a term appears in a document.

$$\mathrm{TF}(t,d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

2. **IDF (Inverse Document Frequency):**
   Measures how important a term is across all documents. Words that appear in many documents get lower scores.

$$\mathrm{IDF}(t) = \log\left(\frac{\text{Number of all documents N}}{\text{Number of documents containing the term } t}\right)$$

	...	the	country	hell	...
...					
425		43	5	1	
426		24	1	0	
427		37	0	3	
...					
df		500	14	7	

tf matrix

	...	the	country	hell	...
...					
425		0	26.0	6.2	
426		0	5.2	0	
427		0	0	18.6	
...					

tf-idf matrix

In [2]:

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
```

In [3]:

```
doc_1 = "Data is the oil of the digital economy"
doc_2 = "Data is a new oil"

data = [doc_1, doc_2]
```

In [4]:

```
tfidf = TfidfVectorizer()
result = tfidf.fit_transform(data) # returns sparce matrix
```

In [5]:

```
df = pd.DataFrame(result.toarray(), columns=tfidf.get_feature_names_out())
df
```

Out[5]:

	data	digital	economy	is	new	of	oil	the
0	0.243777	0.34262	0.34262	0.243777	0.000000	0.34262	0.243777	0.68524
1	0.448321	0.00000	0.00000	0.448321	0.630099	0.00000	0.448321	0.00000

Cosine similarity

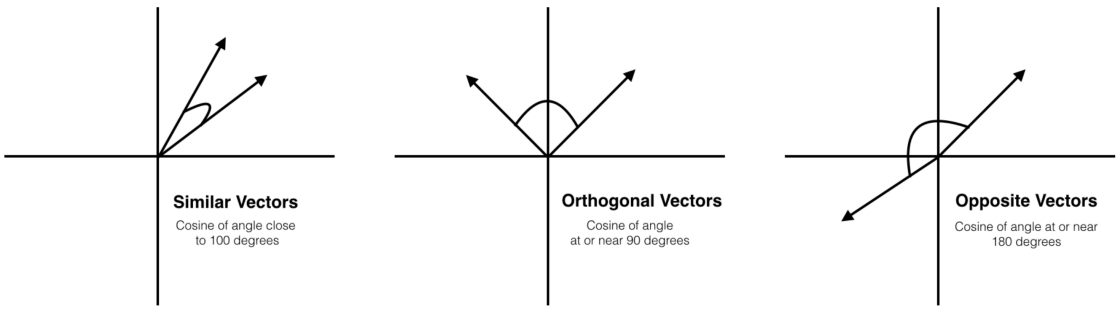
In NLP, Cosine similarity is a metric used to measure how similar the documents are.

$$\text{cosine similarity} = \frac{A \cdot B}{\|A\| \times \|B\|}$$

Where:
$A \cdot B$ = dot product of vectors A and B
$\|A\|$ = magnitude (length) of vector A
$\|B\|$ = magnitude (length) of vector B

Intuition

- If vectors point in the **same direction**, cosine similarity = **1** (maximum similarity).
- If vectors are **orthogonal (90° apart)**, cosine similarity = **0** (no similarity).



In [6]:

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

In [7]:

```
# Sample text data (replace with your own documents)

doc_1 = "Data is the oil of the digital economy"
doc_2 = "Data is a new oil"

data = [doc_1, doc_2]
```

In [8]:

```
tfidf_vectorizer = TfidfVectorizer() # Create a CountVectorizer instance
vector_matrix = tfidf_vectorizer.fit_transform(data) # Fit and transform the documents into numerical vectors
```

In [9]:

```
# Calculate the cosine similarity between the documents
cosine_similarity_matrix = cosine_similarity(vector_matrix)

df_cosine = pd.DataFrame(data=cosine_similarity_matrix, index=data, columns=data)

df_cosine
```

Out[9]:

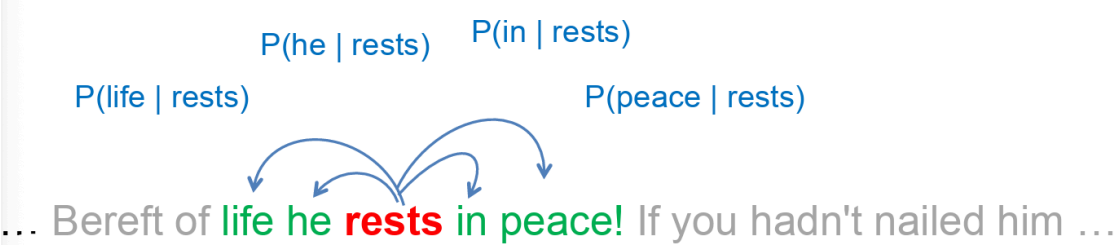
	Data is the oil of the digital economy	Data is a new oil
Data is the oil of the digital economy	1.000000	0.327871
Data is a new oil	0.327871	1.000000

2- What is Word2vec?

Word2Vec consists of models for generating word embedding. These models are two-layer neural networks having one input layer, one hidden layer, and one output layer.

Word2Vec utilizes two architectures :

1. CBOW (Continuous Bag of Words)
2. **Skip Gram**



Run this command in terminal to install

> pip install gensim

We will use fake and real news dataset to do our expirament. You can find the dataset here: https://www.kaggle.com/datasets/clmentbisaillon/fake-and-real-news-dataset?select=True.csv

In [10]:

```
import pandas as pd
import nltk
from pathlib import Path
nltk_data = Path('nltk_data').resolve()
nltk_data.mkdir(exist_ok=True)
nltk.data.path.insert(0, str(nltk_data))
nltk.download('punkt_tab', download_dir=str(nltk_data))
import numpy as np
import gensim

from nltk.tokenize import word_tokenize
```

```
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\ezio9\OneDrive\Desktop\IAU\NLP\NLP lab\LAB
[nltk_data]     5\nltk_data...
[nltk_data]   Unzipping tokenizers\punkt_tab.zip.
```

In [11]:

```
from pathlib import Path
from urllib.request import urlopen
from zipfile import ZipFile
from io import BytesIO

if not Path('data/True.csv').exists():
    Path('data').mkdir(exist_ok=True)
    url = 'https://www.kaggle.com/api/v1/datasets/download/clmentbisaillon/fake-and-real-news-dataset'
    with urlopen(url, timeout=180) as response:
        with ZipFile(BytesIO(response.read())) as archive:
            Path('data/True.csv').write_bytes(archive.read('True.csv'))

df = pd.read_csv('data/True.csv', engine='python', on_bad_lines='skip') #fake-and-real-news-dataset-true.csv
df.head()
```

Out[11]:

	title	text	subject	date
0	As U.S. budget fight looms, Republicans flip t...	WASHINGTON (Reuters) - The head of a conservat...	politicsNews	December 31, 2017
1	U.S. military to accept transgender recruits o...	WASHINGTON (Reuters) - Transgender people will...	politicsNews	December 29, 2017
2	Senior U.S. Republican senator: 'Let Mr. Muell...	WASHINGTON (Reuters) - The special counsel inv...	politicsNews	December 31, 2017
3	FBI Russia probe helped by Australian diplomat...	WASHINGTON (Reuters) - Trump campaign adviser ...	politicsNews	December 30, 2017
4	Trump wants Postal Service to charge 'much mor...	SEATTLE/WASHINGTON (Reuters) - President Donal...	politicsNews	December 29, 2017

In [12]:

```
tokens = []

for i in df['text']:
    token = word_tokenize(i)
    tokens.append(token)
```

In [13]:

```
w2v = gensim.models.Word2Vec(tokens, min_count=1, vector_size=100, window=5, sg=1)
```

In [14]:

```
print("Cosine similarity between 'provide' and 'program' - Skip Gram : ",w2v.wv.similarity('provide', 'program'))
```

```
Cosine similarity between 'provide' and 'program' - Skip Gram :  0.43570235
```

In [15]:

```
print("words that similar to 'program' - Skip Gram : ",w2v.wv.most_similar('program'))
```

```
words that similar to 'program' - Skip Gram :  [('programme', 0.8053586483001709), ('programs', 0.8046126365661621),
('restraints', 0.7031655311584473), ('programmes', 0.6981359720230103), ('arsenal', 0.6772581934928894), ('curbed',
0.6762970089912415), ('entitlement', 0.6718749403953552), ('modernization', 0.6707872748374939), ('curtail',
0.6692838668823242), ('succor', 0.6661202311515808)]
```

Tasks

Task 1: Cosine Similarity

Use the Cosine Similarity method to determine how similar the following sentences are.

'This is the first document.',
'This document is the second document.',
'And this is the third one.',
'Is this the first document?'

In [16]:

```
sentences = [
    'This is the first document.',
    'This document is the second document.',
    'And this is the third one.',
    'Is this the first document?'
]
sentence_vectors = TfidfVectorizer().fit_transform(sentences)
pd.DataFrame(cosine_similarity(sentence_vectors), index=sentences, columns=sentences)
```

Out[16]:

	This is the first document.	This document is the second document.	And this is the third one.	Is this the first document?
This is the first document.	1.000000	0.646926	0.307772	1.000000
This document is the second document.	0.646926	1.000000	0.225240	0.646926
And this is the third one.	0.307772	0.225240	1.000000	0.307772
Is this the first document?	1.000000	0.646926	0.307772	1.000000

Task 2: TF-IDF

Use tf-idf method on the sentences below to determine the important words.

'data science is one of the most important fields of science',
'this is one of the best data science courses',
'data scientists analyze data'

In [17]:

```
documents = [
    'data science is one of the most important fields of science',
    'this is one of the best data science courses',
    'data scientists analyze data'
]
task_tfidf = TfidfVectorizer()
document_vectors = task_tfidf.fit_transform(documents)
tfidf_scores = pd.DataFrame(document_vectors.toarray(), columns=task_tfidf.get_feature_names_out())
display(tfidf_scores)
for index, scores in tfidf_scores.iterrows():
    print(f'Document {index + 1}:')
    print(scores[scores > 0].sort_values(ascending=False).head(3))
```

	analyze	best	courses	data	fields	important	is	most	of	one	science	scientists	the	this
0	0.000000	0.000000	0.000000	0.189526	0.320895	0.320895	0.244049	0.320895	0.488098	0.244049	0.488098	0.000000	0.244049	0.000000
1	0.000000	0.400294	0.400294	0.236420	0.000000	0.000000	0.304434	0.000000	0.304434	0.304434	0.304434	0.000000	0.304434	0.400294
2	0.542701	0.000000	0.000000	0.641055	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.542701	0.000000	0.000000

```
Document 1:
science    0.488098
of         0.488098
most       0.320895
Name: 0, dtype: float64
Document 2:
best       0.400294
courses    0.400294
this       0.400294
Name: 1, dtype: float64
Document 3:
data          0.641055
analyze       0.542701
scientists    0.542701
Name: 2, dtype: float64
```

Word2vec

Task 3:

Download the Simpsons dataset **(simpsons_script_lines.csv)** and apply the preprocessing procedure.
Use the **'spoken_words'** column.

```
def clean_text(text):
    text = text.lower()
    text = re.sub(r"[0-9]", '', text)
    text = re.sub(r"[](),””.’$-]", '', text)
    return text
```

Create a skip gram Word2Vec model as below.

```
Skip_gram_model = gensim.models.Word2Vec(tokens, min_count = 1, vector_size = 100, window = 5, sg = 1)
```

In [18]:

```
import re

simpsons = pd.read_csv('simpsons_script_lines.csv')

def clean_text(text):
    text = text.lower()
    text = re.sub(r"[0-9]", '', text)
    text = re.sub(r"[](,”“.’$-]", '', text)
    return text

simpsons = simpsons.dropna(subset=['spoken_words']).copy()
simpsons['cleaned_words'] = simpsons['spoken_words'].apply(clean_text)
tokens = [word_tokenize(text) for text in simpsons['cleaned_words']]
tokens = [sentence for sentence in tokens if sentence]
Skip_gram_model = gensim.models.Word2Vec(tokens, min_count=1, vector_size=100, window=5, sg=1)
print('Tokenized reviews:', len(tokens))
print('Vocabulary size:', len(Skip_gram_model.wv))
```

```
Tokenized reviews: 131839
Vocabulary size: 44282
```

Task 4

Use: wv.most_similar() method to :

1. Find the words similar to “homer”.
2. Find the words similar to “marge”.
3. Find the words similar to “bart”

In [19]:

```
for word in ['homer', 'marge', 'bart']:
    print(f"Words similar to '{word}':")
    print(Skip_gram_model.wv.most_similar(word))
```

```
Words similar to 'homer':
[('abe', 0.864983856678009), ('marge', 0.8632112741470337), ('bart', 0.8178604245185852), ('bartholomew', 0.8051577806472778),
('grampa', 0.8045763969421387), ('mindy', 0.8002265095710754), ('eliza', 0.7999703884124756), ('lurleen', 0.7868685722351074),
('apu', 0.7853838801383972), ('sweetheart', 0.7828832268714905)]
Words similar to 'marge':
[('homer', 0.8632113337516785), ('abe', 0.8493781089782715), ('lisa', 0.8294623494148254), ('sweetie', 0.8267340064048767),
('neddy', 0.8153856992721558), ('allison', 0.81475430727005), ('sweetheart', 0.8133155107498169), ('becky', 0.8126724362373352),
('apu', 0.8084233403205872), ('greta', 0.804734468460083)]
Words similar to 'bart':
[('milhouse', 0.884640097618103), ('lisa', 0.8681566119194031), ('eliza', 0.838799774646759), ('abe', 0.8369408845901489),
('kirk', 0.8366391658782959), ('sweetie', 0.8323475122451782), ('grampa', 0.8321039080619812), ('jessica', 0.8302655220031738),
('chalmers', 0.8226650357246399), ('sweetheart', 0.8198592662811279)]
```

Task 5

Use the wv.doesnt_match() method to :

1. Find which of 'jimbo', 'milhouse’, and 'kearney’ does not belong to the list.
2. Find the odd one among "nelson", "bart", and "milhouse".
3. Find the odd one among ‘homer', 'patty', and ‘selma'.

Hint: You need to pass the strings as List

In [20]:

```
for words in [
    ['jimbo', 'milhouse', 'kearney'],
    ['nelson', 'bart', 'milhouse'],
    ['homer', 'patty', 'selma']
]:
    print(words, '->', Skip_gram_model.wv.doesnt_match(words))
```

```
['jimbo', 'milhouse', 'kearney'] -> milhouse
['nelson', 'bart', 'milhouse'] -> nelson
['homer', 'patty', 'selma'] -> homer
```